

Prvá desktop appka (Java kód / FXML)

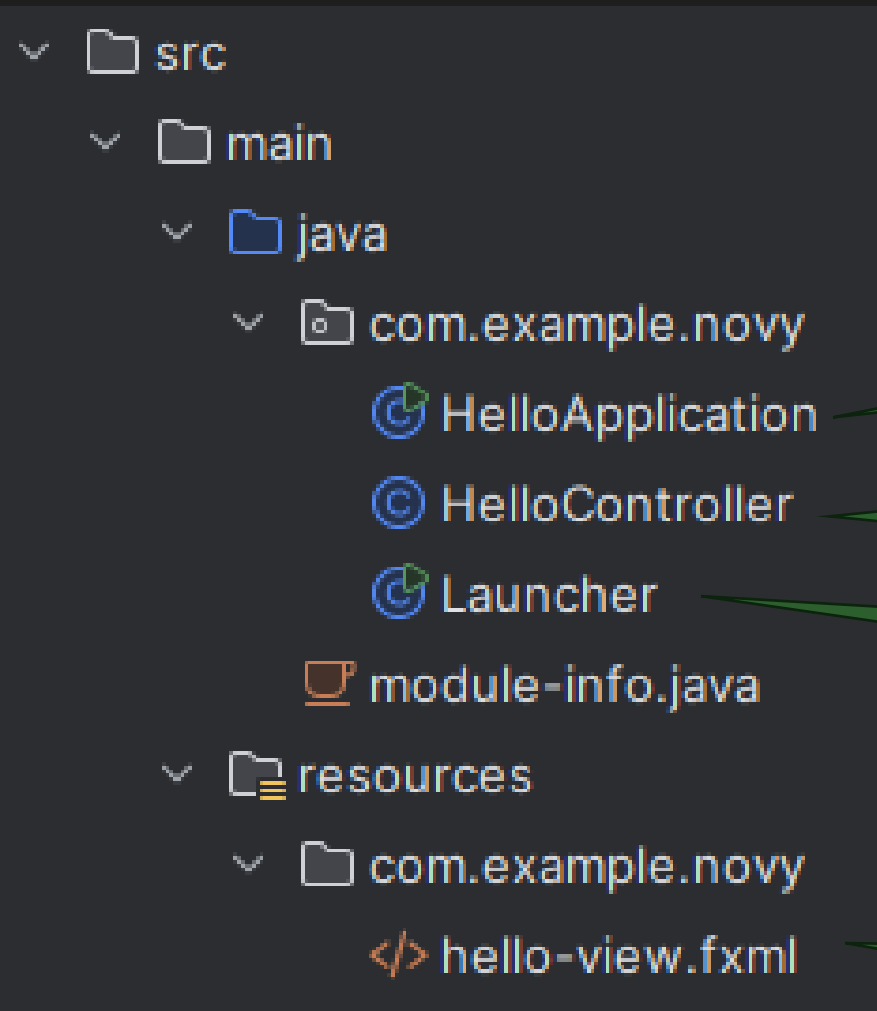
KI/OT/22 - Cvičenie 2

RNDr. Dávid Držík, PhD.



Nová JavaFX aplikácia 1/2

Nová JavaFX aplikácia 2/2



2. Reprezentuje **hlavnú JavaFX aplikáciu**. V metóde start inicializuje primárne okno (Stage), načíta grafické rozhranie z FXML súboru, vytvorí scénu a zobrazí ju používateľovi.

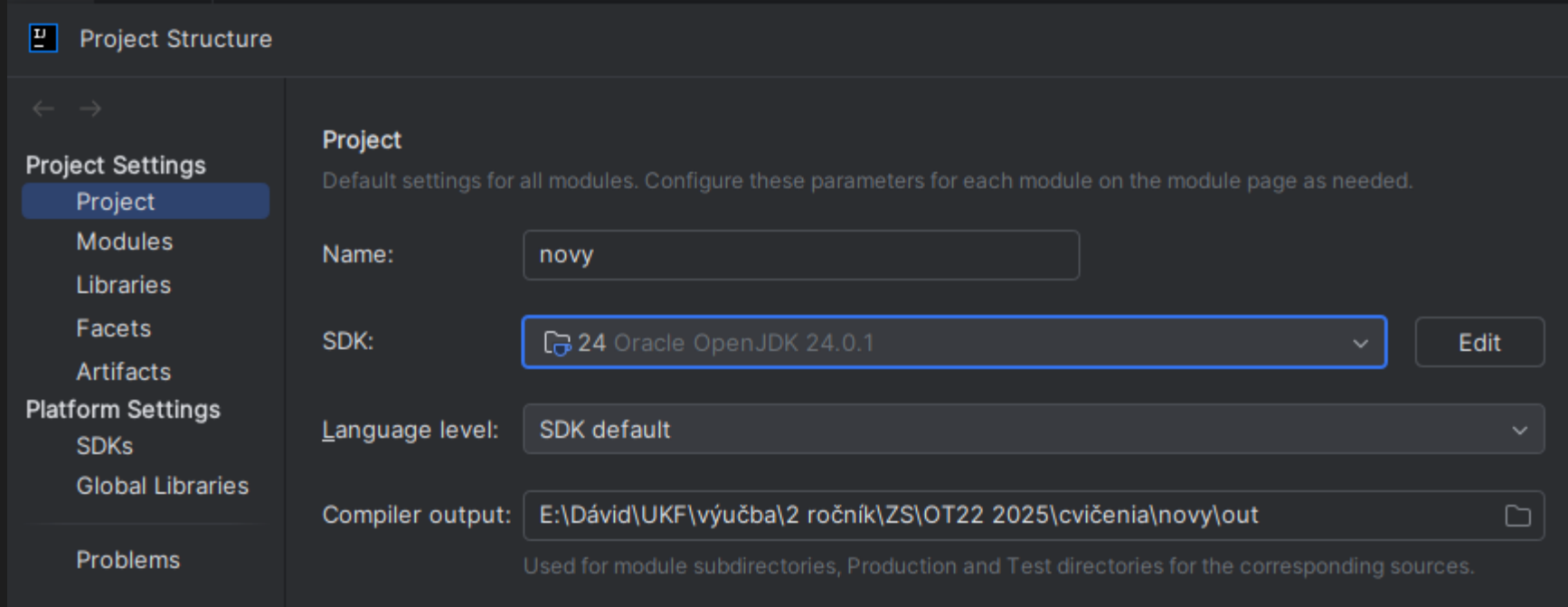
4. Slúži na prepojenie grafického rozhrania definovaného v FXML s aplikačnou logikou.

1. Zabezpečuje spustenie JavaFX aplikácie.

3. Definuje štruktúru používateľského rozhrania aplikácie.

Počiatočné chyby 1/2

- projekt nemá priradený **JDK**, alebo je nastavený len modul bez **SDK**
 - výsledok: IntelliJ nevie kompilovať ani spúšťať



Počiatočné chyby 2/2

- **pom.xml**

- Maven kompiluje projekt ako pre Javu 8, hoci je nainštalované novšie JDK
- výsledok: IntelliJ síce videl JDK 24, ale Maven build padol, pretože verzie nesesedeli

```
<configuration>  
  <source>24</source>  
  <target>24</target>  
</configuration>
```

alebo

```
<configuration>  
  <release>24</release>  
</configuration>
```

Dobrá rada, aj k zápočtu

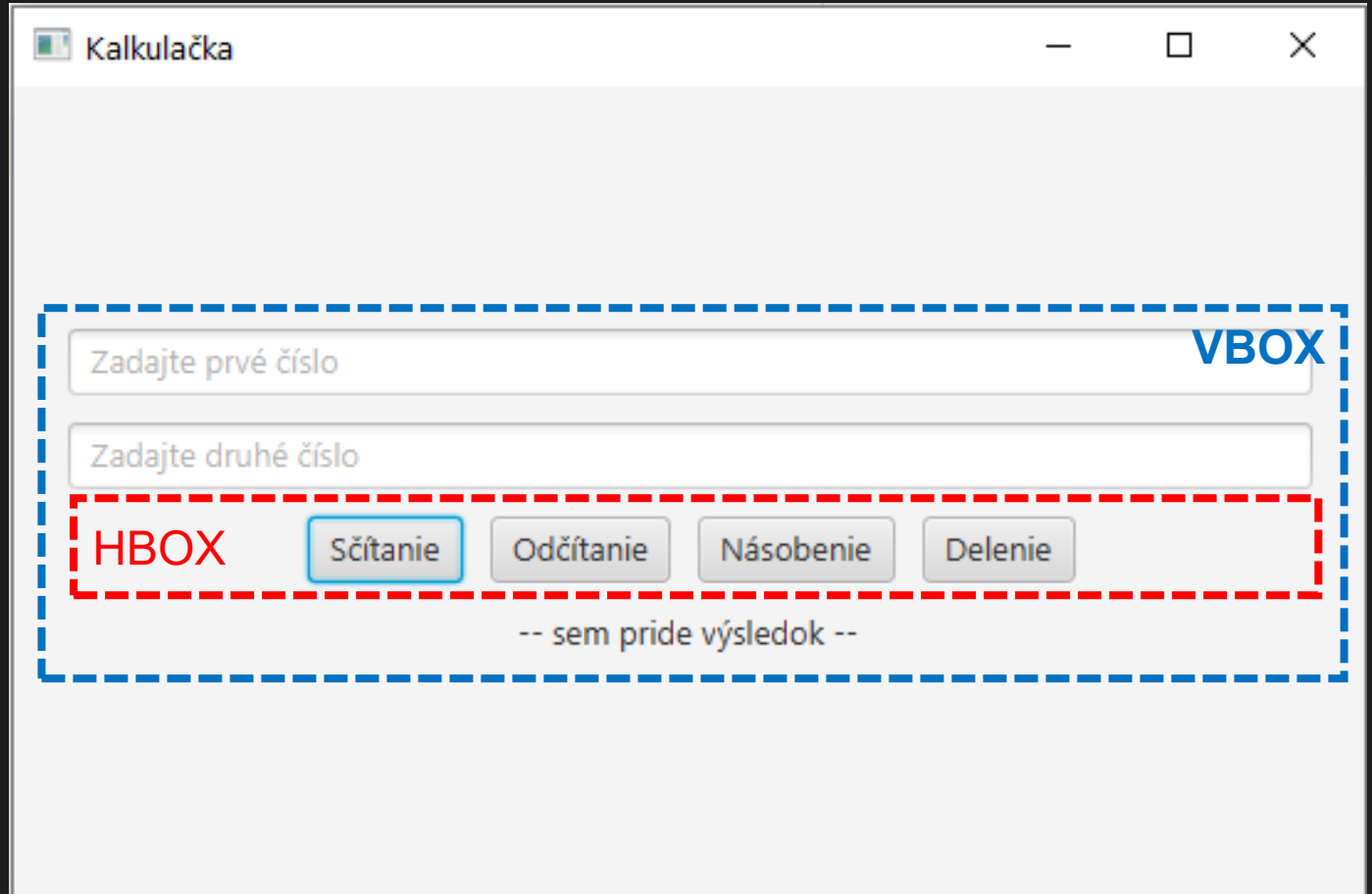
- 01 Ciary
- 02 Pyramida
- 03 NejakylNyZaujímavvyProjekt
- demo

Nevytvárajme za každým novým projektom, ale fungujúcu aplikáciu si odložme a **duplikujme** = šetríme čas.



Úloha 1 – Kalkulačka 1/2

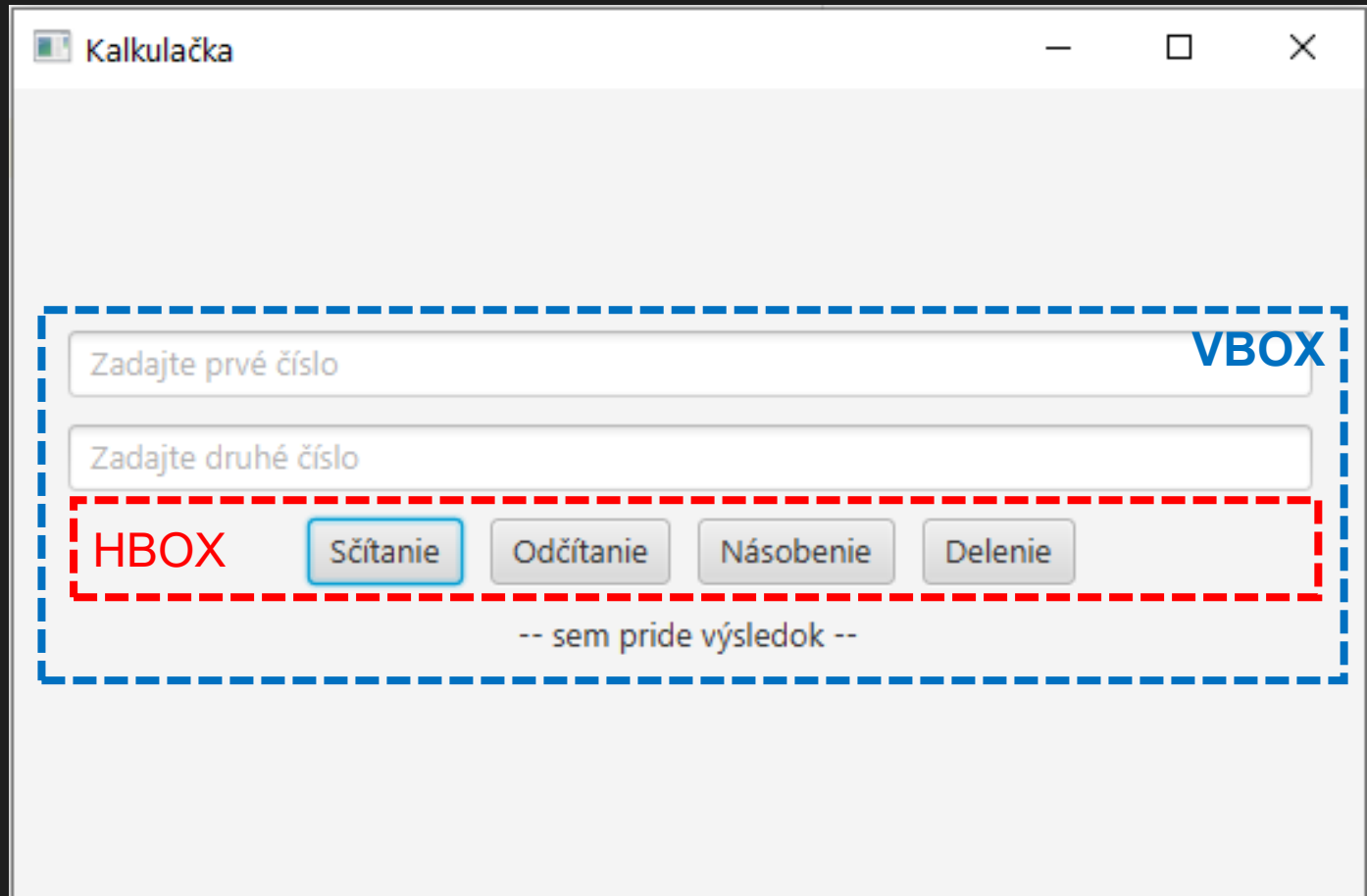
- 2 textové polia
- 4 tlačidlá pre: + - * /
- 1 label pre výsledok
- obsluha udalostí pre každé tlačidlo
- efekty pre tlačidlá, aj výstup



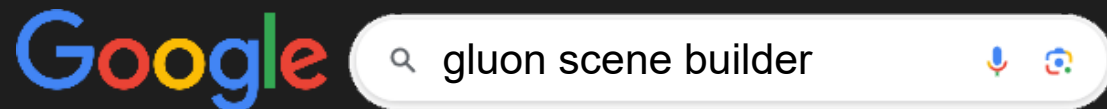


Úloha 1 – Kalkulačka 2/2

- 1 **spoločný handler** pre všetky tlačidlá
 - načítame čísla
 - zistíme o akú operáciu ide
 - `Button sourceButton = (Button) event.getSource();`
 - na základe operácie vypočítame výsledok, ktorý vrátime na label
- nastavíme handler tlačidlám
- aplikujte dizajn z prednášky:
 - `promptText`, `LabelScale`, `shadow`, `cursor HAND`, ...



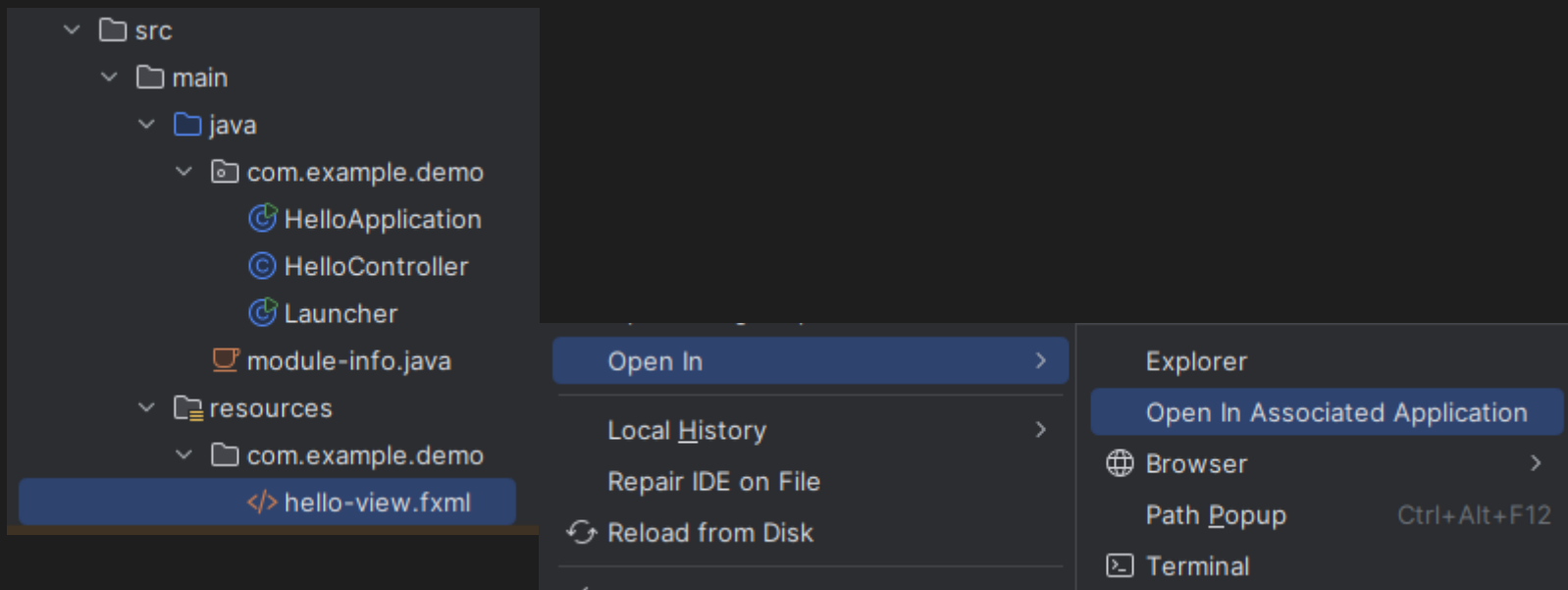
Dá sa to jednoduchšie – Scene Builder



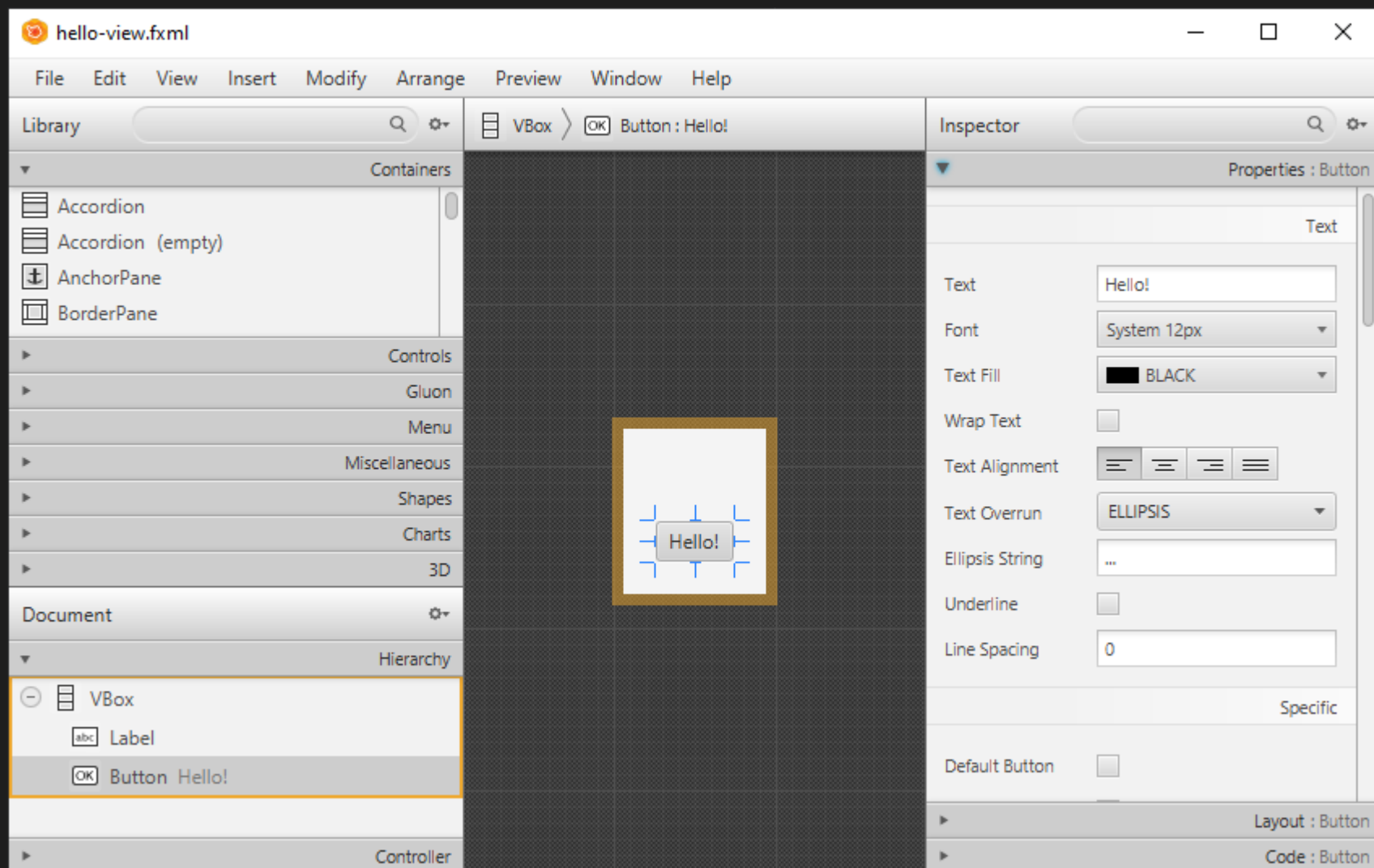
- download Windows Installer → install

- otvoriť fxml súbor
v SceneBuilder

- nájsť exe súbor



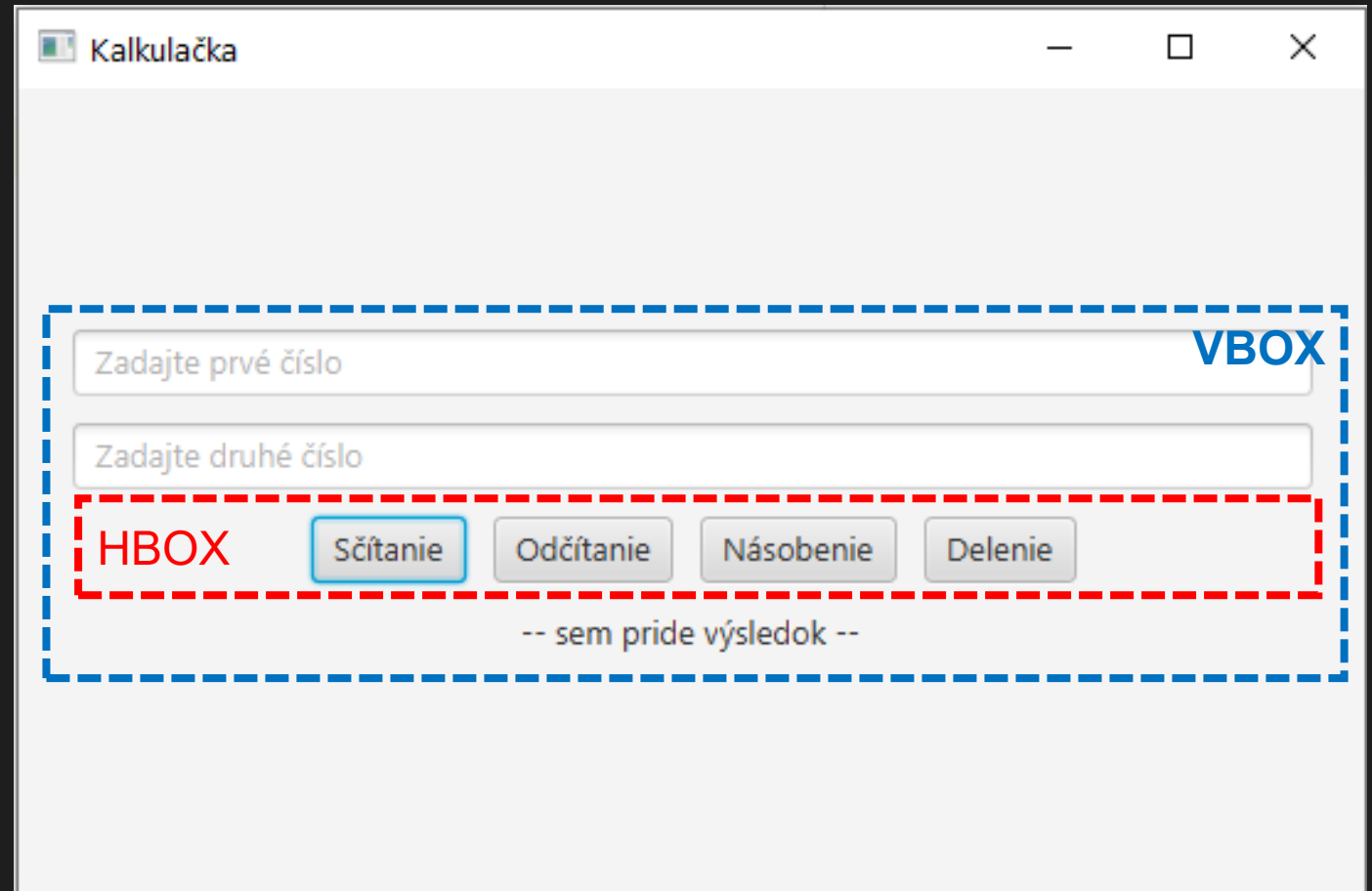
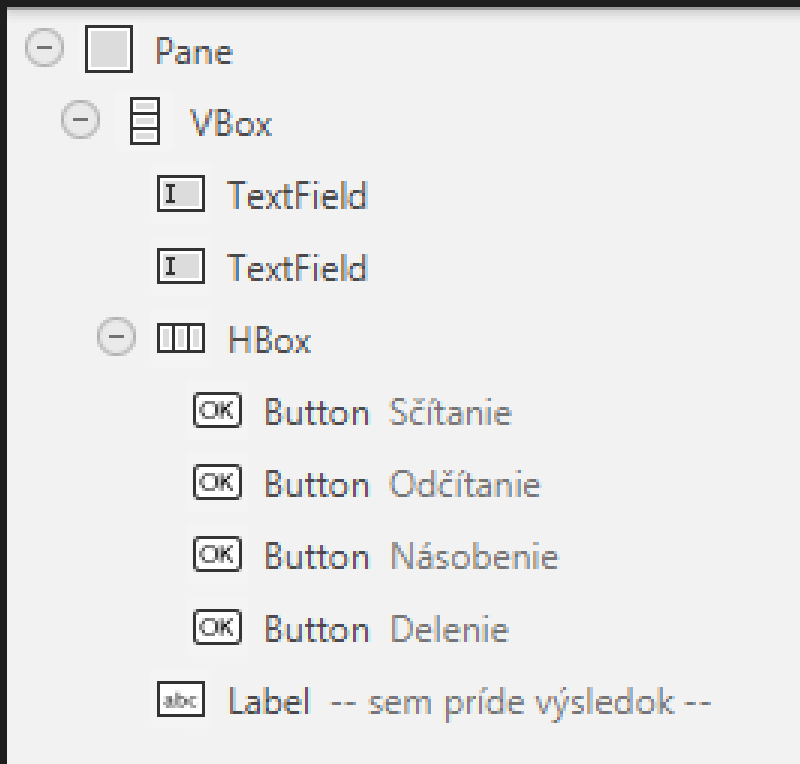
Dá sa to jednoduchšie – Scene Builder





Úloha 2 – Kalkulačka (cez Scene Builder)

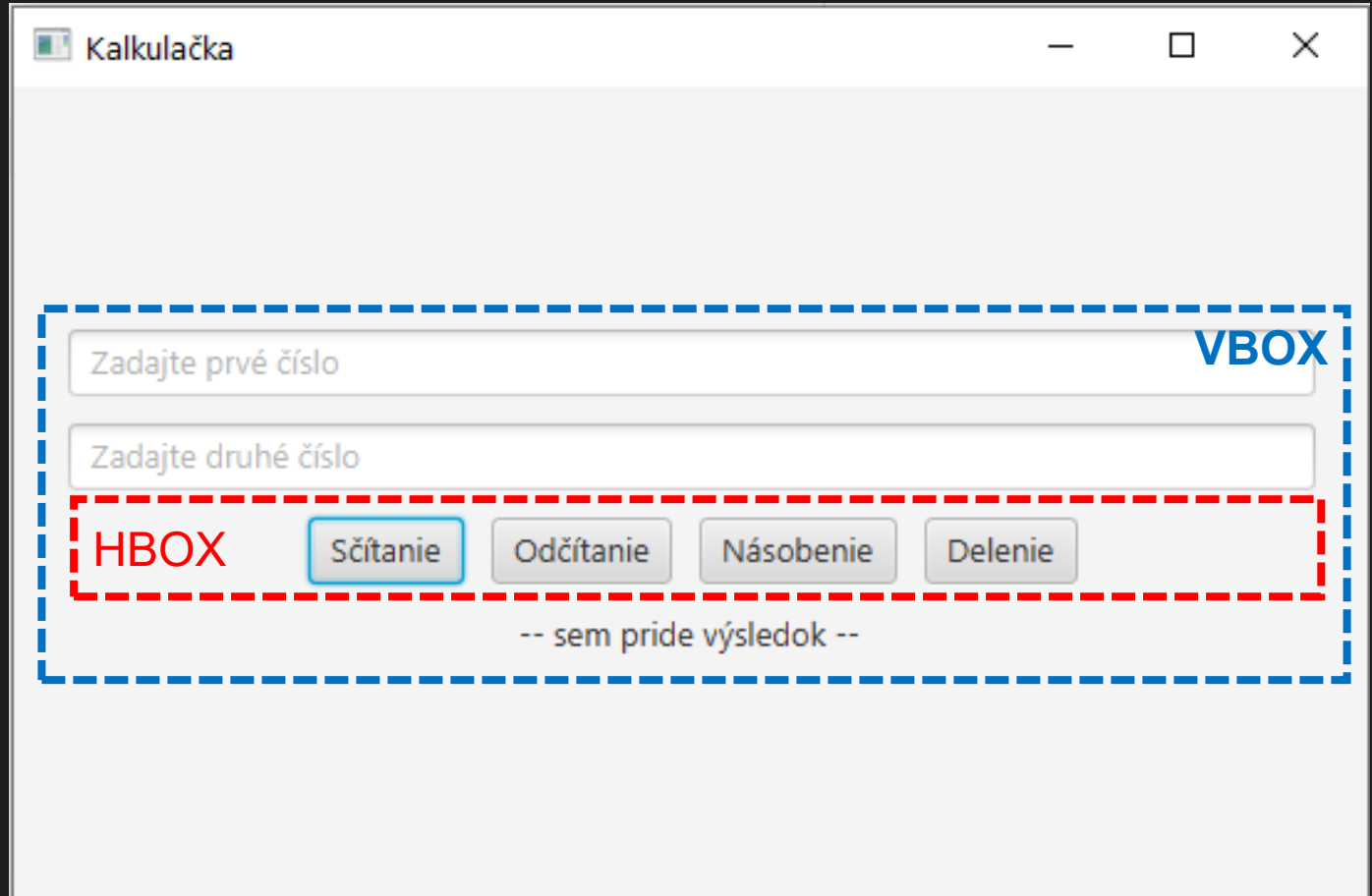
- cez Scene Builder





Úloha 2 – Kalkulačka (cez Scene Builder)

- **fx:controller**
 - definuje, ktorá trieda controllera riadi UI v danom .fxml súbore
- **fx:id**
 - slúži na prepojenie FXML komponentu s premennou v controlleri
- **onAction=#metóda**
 - prepojí akciu z FXML so spúšťanou metódou v controlleri



Metóda initialize() 1/3

- JavaFX ju automaticky zavolá hneď po načítaní FXML súboru a po injektovaní všetkých @FXML prvkov
 - **aplikácia spustí**
 - **FXMLLoader** loader = new **FXMLLoader**(getClass().getResource("hello-view.fxml"));
 - Scene scene = new Scene(loader.load());
 - **FXMLLoader vytvorí inštanciu kontrolera**
 - fx:controller="com.example.demo.**HelloController**"
 - v tomto momente premenné @FXML sú ešte null
 - **FXMLLoader vytvorí všetky UI komponenty zo súboru**
 - napr. Button, TextField, Label podľa XML značiek
 - tieto objekty existujú v pamäti, ale ešte nie sú prepojené s controllerom

Metóda initialize() 2/3

- **FXMLLoader "injectne" komponenty do kontrolera**
 - hľadá vo FXML všetky **fx:id** atribúty: `<Button fx:id="addButton" ... />`
 - a nastaví ich do zodpovedajúcich **polí** v kontroleri: `@FXML private Button addButton;`
- **FXMLLoader zavolá metódu initialize()**
 - keď už sú všetky `@FXML` polia správne priradené
- **scéna sa zobrazí v Stage**
 - až teraz sa **UI ukáže** používateľovi
 - všetky inicializácie z `initialize()` sú už hotové

Metóda initialize() 3/3

- na čo slúži:
 - nastavenie štartovných hodnôt komponentov (text, štýl, viditeľnosť, veľkosť, ...),
 - pridanie listenerov (kliknutie, hover, zmena textu ...),
 - napojenie dát (napr. inicializácia tabuliek, combo-boxov, grafov),
 - príprava efektov (DropShadow, animácie)

Rozdiel: lambda listener vs. FXML handler

• a) Lambda listener v initialize()

- **eventy** sa pripájajú **priamo v kóde**
- **flexibilita**: použitie premenných, podmienok, cyklov
- možnosť dynamicky pripojiť viacero listenerov
- vhodné na aplikovanie rovnakého efektu na viac komponentov
- čistejší FXML súbor (bez atribútov typu `onMouseEntered="#..."`)

```
@FXML
public void initialize() {
    label.setOnMouseEntered(e -> {
        label.setScaleX(1.5);
        label.setScaleY(1.5);
    });
}
```

• b) @FXML handler vo forme metódy

- **event** je definovaný priamo vo **FXML**, controller obsahuje iba metódu
- prehľadné pri jednoduchom UI – vo FXML je ihneď viditeľné, ktorá metóda sa spustí
- **menej flexibilné** pri viacerých komponentoch – vyžaduje pridať atribút k každému prvku
- v controlleri je potrebné vytvoriť viac metód

```
@FXML
public void labelScaleBig(MouseEvent mouseEvent) {
    label.setScaleX(1.5);
    label.setScaleY(1.5);
}
```




Úloha 3 – Kalkulačka (super ultra mega 9000)

- dizajn cez Scene Builder
- uchované medzivýsledky
- možnosť ovládať nie len kurzorom, ale aj klávesmi

